

Compilation II

MU4IN501 – DLP : Développement d'un langage de programmation
Master STL, Sorbonne Université

Antoine Miné

Cours 4

3 octobre 2023

Année 2023–2024



Développement d'un **langage** de programmation :
interprète et **compilateur**.

Par étapes, avec ajout progressif de fonctionnalités.

- **ILP1** : langage de base
 - constantes (entiers, flottants, booléens, chaînes),
 - opérateurs (+, -, ×, ...),
 - appels de primitives (print, ...),
 - blocs locaux (let x = ... in ...),
 - alternatives (if ... then ... else)
- **ILP2** : ajout des boucles, affectations, fonctions globales
- **ILP3** : ajout des exceptions et fonctions de première classe
- **ILP4** : ajout des classes et des objets

Compilation d'ILP1 : deux cours.

La semaine dernière :

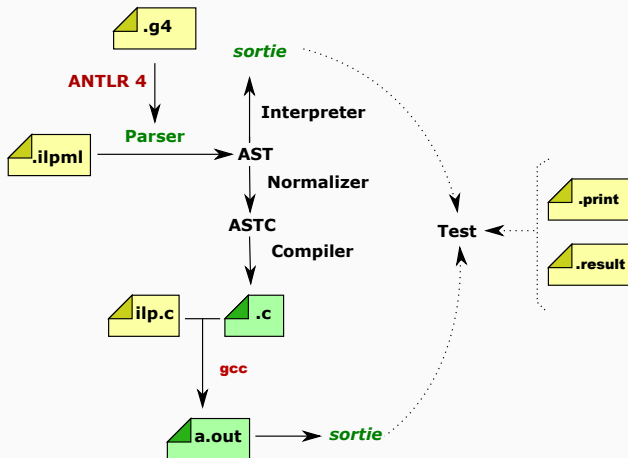
représentation des valeurs et bibliothèque d'exécution.

Cette semaine :

- AST normalisé pour la compilation (ASTC)
- analyse statique et renommage des variables
- génération de code C
- chaîne de compilation

- retour sur les **aspects statiques** et **dynamiques**
- retour sur les **visiteurs**
extension de l'architecture de visiteur ($AST \rightarrow ASTC$)
- mise en œuvre d'une **analyse statique**
et d'une **transformation statique** d'AST
analyse de portée des variables et renommage
- **génération de code**
un programme qui écrit un programme !
- **schéma de compilation**
visualisation schématique de la génération de code
 $\Rightarrow$ **important pour l'examen !**

Schéma général d'ILP

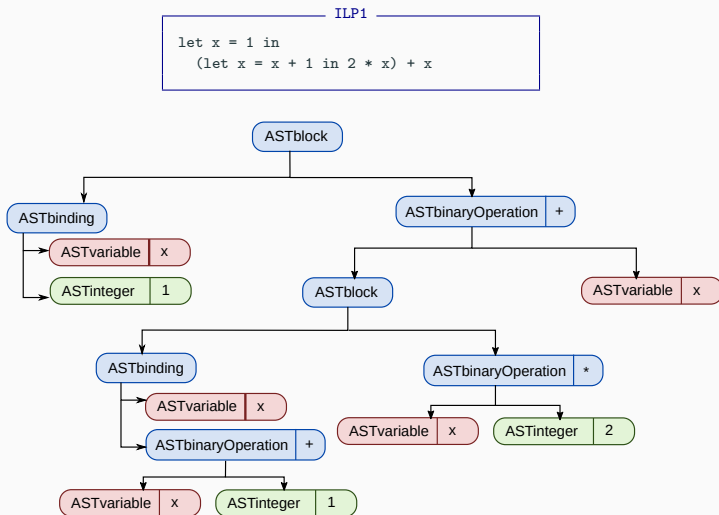


Sources spécifiques au compilateur :

- dans `Java/src/`
 - `com.paracamplus.ilp1.compiler`
point d'entrée, définition des environnements, visiteur de génération de code
 - `com.paracamplus.ilp1.compiler.interfaces`
interfaces d'ASTC
 - `com.paracamplus.ilp1.compiler.ast`
implantations d'ASTC
 - `com.paracamplus.ilp1.compiler.normalizer`
visiteur de conversion AST en ASTC
 - `com.paracamplus.ilp1.compiler.optimizer`
transformation d'ASTC (identité!)
 - `com.paracamplus.ilp1.compiler.test`
tests avec JUnit
- dans `C/`
 - `ilp.[ch]`
bibliothèque d'exécution pour ILP1 à ILP4
 - `gc-8.2.2`
ramasseur de miettes Boehm GC

ASTC : l'AST normalisé

Identification des variables et des portées (1/2)

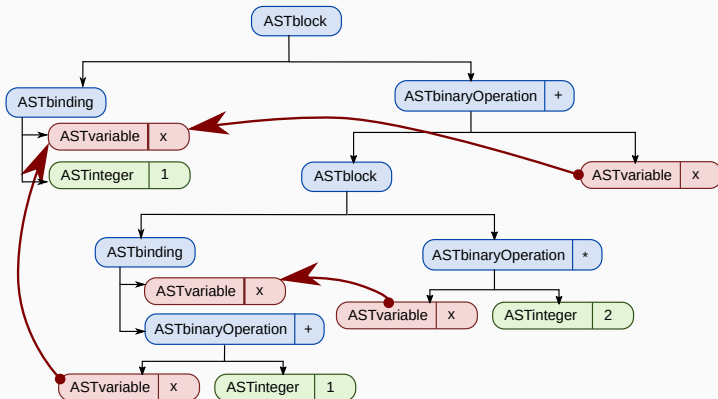


Problème : définition et utilisation de variables de même nom (**x**).

Identification des variables et des portées (2/2)

ILP1

```
let x = 1 in
  (let x = x + 1 in 2 * x) + x
```



Solution : lier chaque utilisation d'une variable à sa définition.

But : résoudre la portée statiquement.

Classification :

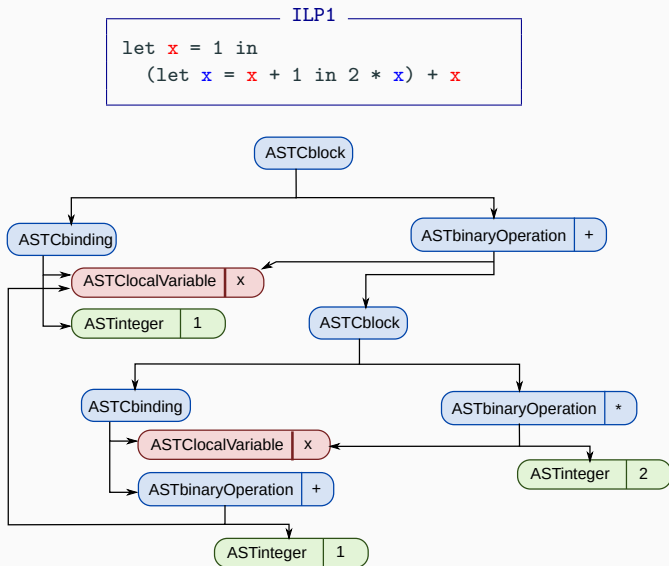
Distinction du type de portée :

- `ASTClocalVariable` : variable locale (introduite par `let`)
- `ASTCglobalVariable` : variable globale (en ILP1, les primitives)
- `ASTCvariable` : père de `ASTClocalVariable` et `ASTCglobalVariable`

Partage et identification :

- chaque définition de variable, dans un `let` (`ASTCbinding` dans l'AST) génère un objet `ASTClocalVariable` unique
- chaque utilisation de variable référence l'`ASTClocalVariable` du `let` qui la définit (pas de création d'objet `ASTClocalVariable` pour le référencement)
- deux références à la même variable sont représentées par la même adresse ; l'**identité** de variable est testable en Java avec `==`

⇒ l'ambiguïté des noms de variable est résolue par l'**adresse d'objet Java** !



ASTC : renommage des variables

ILP après renommage

```
let x1 = 1 in  
  (let x2 = x1 + 1 in 2 * x2) + x1
```

C généré (simplifié)

```
ILP_Object ilp_program ()  
{  
  ILP_Object ilptmp1;  
  ILP_Object x1 = ILP_Integer2ILP (1);  
  {  
    ILP_Object x2 =  
      ILP_Plus (x1, ILP_Integer2ILP (1));  
    ilptmp1 = ILP_Times (ILP_Integer2ILP (2), x2);  
  }  
  return ILP_Plus (ilptmp1, x1);  
}
```

Dans l'ASTC, un **nom unique** est donné à chaque variable **ASTCvariable** :

- le nom unique est **utilisé comme nom de variable C** dans la génération de code pour éviter les ambiguïtés
- facilite le traitement et l'affichage des variables
on peut utiliser **Set<IASTCvariable>**, **Map<IASTCvariable,C>**
ou **Set<String>** et **Map<String,C>**
- on en profite pour réécrire les caractères non autorisés en C, e.g. « : » → « _3a »

Si on se limite aux variables locales immuables d'ILP1
la détermination de portée semble facile, mais :

- ILP2 ajoutera les **variables globales** utilisateur,
l'affectation
et la déclaration de **fonctions utilisateur**
- ILP3 ajoutera les **fonctions de première classe**

ILP2

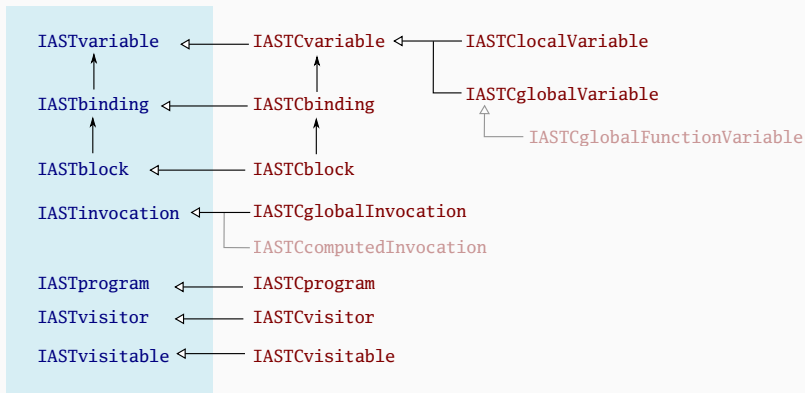
```
g = 12;  
function f(l)  
  (g = g + 1; g + l);  
let x = g + 1 in f(x)
```

ILP3

```
x = 1;  
function f(y)  
  let z = 2 in  
  lambda (t) (x + y + z + t)
```

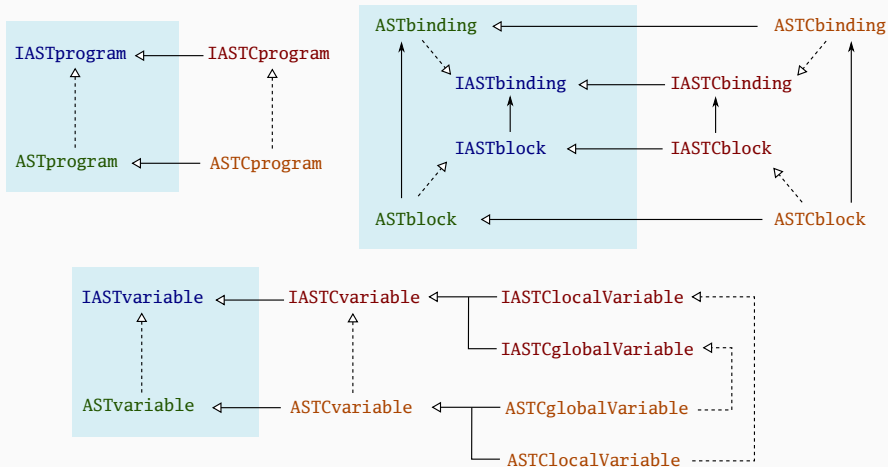
⇒ nous mettons dès maintenant en place un mécanisme général de portée
qui nous servira par la suite.

Architecture des interfaces : IASTC



- spécialisation des interfaces des variables, ou contenant des variables
- mise à jour du visiteur
- les autres interfaces d'IAST sont reprises inchangées
- les interfaces de fonctions globales et d'appels non-primitifs ne seront utiles que pour ILP2

Architecture des implantations : ASTC



Héritage de classes d'AST et implantation d'interfaces IASTC héritant d'IAST.

seule l'implantation des méthodes ajoutées entre l'IAST et l'IASTC est nécessaire !

Les variables de l'ASTC

IASTCvariable.java

```
public abstract interface IASTCvariable extends IASTvariable, IASTCvisitable {  
    boolean isMutable();  
    void setMutable();  
}
```

ASTCvariable.java

```
public class ASTCvariable extends ASTvariable {  
  
    public ASTCvariable(String name) {  
        super(name);  
        this.mutable = false;  
    }  
    private boolean mutable;  
    public boolean isMutable() { return mutable; }  
    public void setMutable() { mutable = true; }  
}
```

- **IASTCvariable** ajoute une propriété « modifiable », non encore utilisée
- **IASTClocalVariable** ajoute une propriété « close », utile en ILP3
- **IASTCglobalVariable** n'ajoute rien
- ce sont surtout des **interfaces marqueur**
⇒ on peut tester qu'une **ASTCvariable** est une variable locale avec « **instanceof ASTClocalVariable** »

IASTCblock.java

```
public interface IASTCblock extends IASTblock {  
    interface IASTCbinding extends IASTblock.IASTbinding  
    { @Override IASTClocalVariable getVariable(); }  
    @Override IASTCbinding[] getBindings();  
}
```

ASTCblock.java

```
public class ASTCblock extends ASTblock implements IASTCblock {  
  
    public static class ASTCbinding extends ASTblock.ASTbinding implements IASTCbinding {  
        public ASTCbinding (IASTvariable variable, IASTexpression initialisation)  
        { super(variable, initialisation); }  
  
        @Override public IASTClocalVariable getVariable ()  
        { return (IASTClocalVariable) super.getVariable(); }  
    }  
  
    public ASTCblock (IASTCbinding[] binding, IASTexpression body ) { super(binding, body); }  
  
    @Override public IASTCbinding[] getBindings() {return (IASTCbinding[])super.getBindings();}  
  
    public <...> Result accept(IASTCvisitor<...> visitor, Data data) throws Anomaly  
    { return visitor.visit(this, data); }  
}
```

Il ne s'agit en fait que de **mettre à jour les types** pour retourner des **IASTCvariable**.

Les programmes de l'ASTC

IASTCprogram.java

```
public interface IASTCprogram extends IASTprogram
{
    Set<IASTCglobalVariable> getGlobalVariables();
    void setGlobalVariables(Set<IASTCglobalVariable> gvs);
}
```

ASTCprogram.java

```
public class ASTCprogram extends ASTprogram implements IASTCprogram
{
    public ASTCprogram (IASTexpression expression) {
        super(expression);
        this.globalVariables = new HashSet<>();
    }
    protected Set<IASTCglobalVariable> globalVariables;

    @Override public Set<IASTCglobalVariable> getGlobalVariables() {
        { return globalVariables; }
    }

    @Override public void setGlobalVariables(Set<IASTCglobalVariable> gvs)
    { globalVariables = gvs; }
}
```

L'ASTC ajoute aux programmes de l'AST un attribut pour maintenir l'ensemble des variables globales. Cela sera surtout utile à partir d'ILP2...

INormalizationFactory.java

```
public interface INormalizationFactory
{
    IASTCvariable      newVariable (String name) throws CompilationException;
    IASTClocalVariable newLocalVariable (String name);
    IASTCglobalVariable newGlobalVariable (String name);
    IASTCexpression     newSequence (IASTexpression[] asts);
    ...
}
```

Comme pour l'AST, on utilise un **pattern fabrique** (*Factory*).

- ne pas appeler `new ASTCvariable(...)` directement
- mais plutôt prendre en argument un `INormalizationFactory factory` et appeler `factory.newVariable(...)`

⇒ on ne dépend que de l'interface `IASTCvariable`
et pas de l'implantation `ASTCvariable`, qui peut évoluer

L'interface **n'étend pas** `IASTfactory`

cela évite d'hériter de méthodes construisant des nœuds **interdits** dans l'ASTC

e.g. : l'ASTC ne doit pas contenir d'`ASTvariable`, d'`ASTblock`, etc., il faut la version ASTC;
par contre, il peut contenir des `ASTsequence`, nœud AST non redéfini en ASTC...

NormalizationFactory.java

```
public class NormalizationFactory implements INormalizationFactory {

    protected AtomicInteger count;

    public NormalizationFactory() { this.count = new AtomicInteger(0); }

    @Override public IASTCvariable newVariable(String name)
    throws CompilationException {
        throw new CompilationException("Uncategorized variable " + name);
    }

    @Override public IASTClocalVariable newLocalVariable(String name) {
        String newName = name + count.incrementAndGet();
        return new ASTClocalVariable(newName);
    }

    @Override public IASTCglobalVariable newGlobalVariable(String name) {
        return new ASTCglobalVariable(name);
    }

    @Override public IASTexpression newSequence(IASTexpression[] asts) {
        return new ASTsequence(asts);
    }

    ...
}
```

La fabrique s'occupe aussi de **renommer de manière unique les variables locales** à leur création grâce à un compteur **count**.

IASTCvistor.java

```
public interface IASTCvisitor<Result, Data, Anomaly extends Throwable>
    extends IASTvisitor<Result, Data, Anomaly> {
    Result visit(IASTCglobalVariable iast, Data data) throws Anomaly;
    Result visit(IASTClocalVariable iast, Data data) throws Anomaly;
    Result visit(IASTCvariable iast, Data data) throws Anomaly;
    Result visit(IASTCcomputedInvocation iast, Data data) throws Anomaly;
}
```

IASTCvisitable.java

```
public interface IASTCvisitable extends IASTvisitable {
    <Result, Data, Anomaly extends Throwable>
    Result accept(IASTCvisitor<Result, Data, Anomaly> visitor,
        Data data) throws Anomaly;
}
```

L'architecture de visiteur d'IAST est étendue et spécialisée à l'IASTC.

Attention, il y a une méthode visite pour des interfaces en relation d'héritage :

IASTvariable ← IASTCvariable ← IASTClocalVariable

IASTinvocation ← IASTCglobalInvocation

bien s'assurer dans le visiteur que la bonne méthode est appelée ! (cf. plus loin)

Normalisation de l'AST

L'environnement de normalisation

INormalizationEnvironment.java

```
public interface INormalizationEnvironment {  
    INormalizationEnvironment extend(IASVariable oldv, IASVariable newv);  
    IASVariable renaming(IASVariable variable) throws NoSuchLocalVariableException;  
}
```

L'environnement est une **pile fonctionnelle** associant :

- à chaque variable ILP avec le nom original du programme ILP
- une variable **IASVariable** unique, ayant un nom unique

Deux opérations :

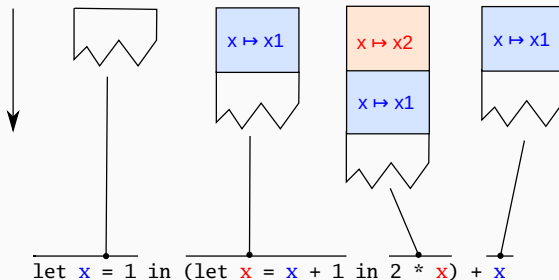
- **extend** : retourne une pile avec une **association supplémentaire au sommet** ; la pile originelle (**this**) reste **inchangée**
- **renaming** : retrouve l'association au plus haut de la pile ayant le **même nom** de variable que celui en argument

```
if (variable.getName().equals(otherVariable.getName())) return newvariable
```

Il est implémenté par une liste chaînée dans **NormalizationEnvironment**.

Implantation similaire à **LexicalEnvironment utilisé par l'interprète !**

Pile des portées : illustration



Parcours de l'AST pour renommer les variables :

- entrée dans un bloc `let` : empilement avec `extend`
- référencement de variable dans le bloc : recherche avec `renaming`
- sortie du bloc : restauration de la pile initiale

Normalizer.java (début)

```
public class Normalizer implements
IASTvisitor<IASTexpression, INormalizationEnvironment, CompilationException> {

    protected final INormalizationFactory factory;

    public IASTCprogram transform(IASTprogram program) throws CompilationException {
        IASTexpression body = program.getBody();
        IASTexpression newbody = body.accept(this, NormalizationEnvironment.EMPTY);
        return factory.newProgram(newbody);
    }

    @Override public IASTexpression visit(IASTinteger iast, INormalizationEnvironment env)
    throws CompilationException
    { return iast; }

    ...
}
```

- point d'entrée : `transform IASTprogram → IASTCprogram`
- les feuilles constantes sont inchangées (`IASTinteger`, etc.)

Visiteur de normalisation (2/3)

Normalizer.java (suite)

```
@Override public IASTExpression visit(IASTBinaryOperation iast, ... env) ... {
    IASTExpression left = iast.getLeftOperand().accept(this, env);
    IASTExpression right = iast.getRightOperand().accept(this, env);
    return factory.newBinaryOperation(iast.getOperator(), left, right);
}

@Override public IASTExpression visit(IASTBlock iast, ... env) ... {
    INormalizationEnvironment newenv = env;
    IASTbinding[] bindings = iast.getBindings();
    IASTCblock.IASTCbinding[] newbindings = new IASTCblock.IASTCbinding[bindings.length];
    for ( int i=0 ; i<bindings.length ; i++ ) {
        IASTExpression expr = bindings[i].getInitialisation();
        IASTExpression newexpr = expr.accept(this, env);
        IASTvariable variable = bindings[i].getVariable();
        IASTvariable newvariable = factory.newLocalVariable(variable.getName());
        newenv = newenv.extend(variable, newvariable);
        newbindings[i] = factory.newBinding(newvariable, newexpr);
    }
    IASTExpression newbody = iast.getBody().accept(this, newenv);
    return factory.newBlock(newbindings, newbody);
}
```

- appel **récuratif** sur les sous-expressions et **reconstruction** des nœuds
- **let x = e1 in e2** : appel récuratif sur **e1** avec l'environnement courant et création d'un nouvel environnement (avec **x**) pour l'appel sur **e2**

Normalizer.java (fin)

```
@Override public IASTvariable visit(IASTvariable iast, ... env) ... {
    try {
        return env.renaming(iast);
    }
    catch (NoSuchLocalVariableException exc) {
        for ( IASTvariable gv : globalVariables )
            if ( iast.getName().equals(gv.getName()) ) return gv;
        IASTvariable gv = factory.newGlobalVariable(iast.getName());
        globalVariables.add(gv);
        return gv;
    }
}
```

Pour les variables :

- recherche d'une **variable locale** dans l'environnement de normalisation
⇒ `env.renaming(iast)`
- si non trouvée, la variable est supposée **globale**
⇒ `newGlobalVariable`

il peut aussi s'agir d'une primitive

note : l'attribut `globalVariables` de `Normalizer` est calculé, mais pas utilisé

Les environnements de compilation

L'environnement global contient :

- la liste des variables primitives (`pi`)
- la liste des fonctions primitives et leur arité (`print`)
- l'**association** entre le **nom ILP** (`print`) et le **nom C** des primitives (`ILP_print`)

Il existe un seul environnement global :

- de classe `GlobalVariableEnvironment`
d'interface `IGlobalVariableEnvironment`
- attribut `globalVariableEnvironment` de la classe `Compiler`
- initialisé par la méthode statique de `GlobalVariableStuff`
- passé au constructeur de `Compiler`

Attention : contrairement à ce que le nom laisse entendre, l'environnement ne contiendra que les primitives ; il ne sera pas enrichi avec les variables et fonctions globales utilisateur dans la suite d'ILP !

L'environnement des primitives : implantation

`IPrimitive.java`

```
public interface IPrimitive {  
    String getName();  
    String getCName();  
    int getArity();  
}
```

`IGlobalVariableEnvironment.java`

```
public interface IGlobalVariableEnvironment {  
    void addGlobalVariableValue (String variableName, String cName);  
    void addGlobalFunctionValue (IPrimitive primitive);  
    boolean isPrimitive(IASVariable variable);  
    IPrimitive getPrimitiveDescription(IASVariable variable);  
    String getCName (IASVariable variable);  
    boolean contains(IASVariable var);  
}
```

- ajouter une variable primitive, avec un nom ILP et un nom C
- ajouter une fonction primitive, avec le nom C et l'arité
- retrouver le nom C et l'arité (`getCName`, `getPrimitiveDescription`)
- déterminer si une variable ou fonction primitive existe (`contains`) et s'il s'agit d'une fonction (`isPrimitive`)

Les implantations de ces interfaces sont de simples conteneurs (`Map<String, IPrimitive>`).

GlobalVariableStuff.java

```
public class GlobalVariableStuff {  
  
    public static void fillGlobalVariables (IGlobalVariableEnvironment env) {  
        env.addGlobalVariableValue("pi", "ILP_PI");  
        env.addGlobalFunctionValue( new Primitive("print",      "ILP_print", 1) );  
        env.addGlobalFunctionValue( new Primitive("newline",    "ILP_newline", 0) );  
        env.addGlobalFunctionValue( new Primitive("to_string",  "ILP_to_string", 1) );  
        env.addGlobalFunctionValue( new Primitive("type_of",    "ILP_type_of", 1) );  
    }  
}
```

À enrichir quand la bibliothèque d'exécution est étendue pour permettre aux programmes ILP d'appeler de nouvelles primitives. . .

L'environnement des opérateurs

IOperatorEnvironment.java

```
public interface IOperatorEnvironment {  
    String getUnaryOperator (IASTOperator operator) throws CompilationException;  
    String getBinaryOperator (IASTOperator operator) throws CompilationException;  
    void addUnaryOperator (String operator, String cOperator) throws CompilationException;  
    void addBinaryOperator (String operator, String cOperator) throws CompilationException;  
}
```

OperatorStuff.java

```
public class OperatorStuff {  
    public static void fillUnaryOperators (IOperatorEnvironment env) throws ... {  
        env.addUnaryOperator("-", "ILP_Opposite");  
        env.addUnaryOperator("!", "ILP_Not");  
    }  
  
    public static void fillBinaryOperators (IOperatorEnvironment env) throws ... {  
        env.addBinaryOperator("+", "ILP_Plus");  
        env.addBinaryOperator("*", "ILP_Times");  
        env.addBinaryOperator("/", "ILP_Divide");  
        ...  
    }  
}
```

Idem pour les opérateurs, unaires et binaires.

IASTOperator = Inamed est une interface proposant getName
et getMangledName qui a le même rôle que getCName (méthode d'interface default)

Génération de code

Rappels :

- types ILP
⇒ type C générique `ILP_Object` pouvant contenir toute valeur ILP
- variable ILP
⇒ variable C (de type `ILP_Object`)
- séquence ILP
⇒ séquence C
- bloc ILP
⇒ bloc C, avec portée locale des variables
- `if then else` ILP
⇒ `if else` C
- expression retournant une valeur en ILP
⇒ affectation de la valeur de retour dans une variable temporaire en C
- programme retournant une valeur en ILP
⇒ fonction `ilp_program` retournant une valeur

Redirection d'expressions

Les expressions ILP retournent toujours un résultat
y compris dans des contextes où le C ne retourne pas de valeur ! (`if then else`).

⇒ on passe **en argument** à la compilation d'une expression
une indication sur **quoi faire du résultat** de l'expression
(mécanisme de *continuation*, ou d'*inversion du contrôle*)

Notation : $\xrightarrow{d}$
`expr`

- génération du code C de l'expression ILP `expr`
- en précisant la destination ***d*** du résultat calculé par l'expression

La destination $\rightarrow d$ peut être :

- $\rightarrow$ **return** : retourner de la fonction avec cette valeur
- $\rightarrow$ **(x =)** : stocker la valeur dans la variable C de nom `x`
- $\rightarrow$ **(void)** : ignorer la valeur

Exemples : $\xrightarrow{x=}$
`2` $\rightarrow$ `x = ILP_Integer2ILP(2);`

$\xrightarrow{\text{return}}$
`true` $\rightarrow$ `return ILP_TRUE;`

Implantation des destinations

IDestination.java

```
public abstract interface IDestination {  
    String compile ();  
}
```

ReturnDestination.java

```
public class ReturnDestination implements IDestination {  
    public static final ReturnDestination RETURN_DESTINATION = new ReturnDestination();  
    @Override public String compile() { return "return "; }  
}
```

AssignDestination.java

```
public class AssignDestination implements IDestination {  
    private final IASTvariable variable;  
    public AssignDestination (IASTvariable variable) { this.variable = variable; }  
    @Override public String compile() { return variable.getMangledName() + " = "; }  
}
```

`compile()` indique le **code C** à **générer avant** celui de l'expression afin d'exploiter son résultat :

- `ReturnDestination` : affiche `return`
- `AssignDestination` : affiche `var =`
- `VoidDestination` : affiche `(void)` (évite un avertissement du compilateur C car la valeur n'est pas utilisée)
- `NoDestination` : aucun affichage

Compiler.java (extrait)

```
public class Compiler ... {

    public static class Context {
        public IDestination destination;
        public static AtomicInteger counter = new AtomicInteger(0);

        public Context (IDestination destination) { this.destination = destination; }

        public IASTvariable newTemporaryVariable () {
            int i = counter.incrementAndGet();
            return new ASTvariable("ilptmp" + i);
        }

        public Context redirect (IDestination d) {
            if ( d == destination ) return this;
            else return new Context(d);
        }
    }

    ...
}
```

L'objet **Context** est utilisé par **Compiler** pendant la génération de code pour :

- maintenir la **destination** de l'expression en cours de compilation initialement, c'est un **ReturnDestination** (retour de `ilp_program`)
- générer des **variables temporaires**, grâce à un compteur **static**

Compiler.java (extrait)

```
public class Compiler
implements IASTCvisitor<Void, Compiler.Context, CompilationException> {

    public Void visit(IASTCprogram iast, Context context)
    throws CompilationException {
        emit(cProgramPrefix);
        ... // partie spécifique à ILP2 omise
        emit(cBodyPrefix);
        Context cr = context.redirect(ReturnDestination.RETURN_DESTINATION);
        iast.getBody().accept(this, cr);
        emit(cBodySuffix);
        emit(cProgramSuffix);
        return null;
    }

    protected String cBodyPrefix = "\n"
        + "ILP_Object ilp_program () \n"
        + "{ \n";
    protected String cBodySuffix = "\n"
        + "} \n";
    ...
}
```

Le générateur de code fait un parcours récursif de l'AST (comme l'interprète).

Note : `emit` fait un `append` dans un objet `Write`.

Compiler.java (extrait)

```
@Override public Void visit(IASInteger iast, Context context)
throws CompilationException {
    emit(context.destination.compile());
    emit("ILP_Integer2ILP(");
    emit(iast.getValue().toString());
    emit("); \n");
    return null;
}

@Override public Void visit(IASBoolean iast, Context context)
throws CompilationException {
    emit(context.destination.compile());
    if ( iast.getValue() ) {
        emit("ILP_TRUE");
    } else {
        emit("ILP_FALSE");
    }
    emit("); \n");
    return null;
}

...
```

- affichage de la destination en préfixe : `context.destination.compile()`
- affichage de la constante au format C : `ILP_Integer2ILP(...)`, `ILP_TRUE`, etc.

Le code Java qui génère le code C est difficile à lire !

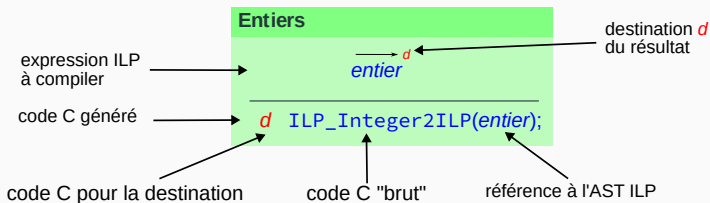
instructions `emit`, découpage en lignes, caractères d'échappement, etc.

⇒ introduction du **schéma de compilation**

- associe à une expression $\overrightarrow{\text{expr}}^d$ un code C « à trous »
- les « trous » sont des appels à des fonctions Java en particulier des **appels récurifs** au générateur de code ! (cf plus loin)
- présentation plus proche du code généré que du générateur de code (facile à déduire) donne une meilleur idée du code C correspondant (sans les `emit`, les chaînes, l'échappement, etc.)
- certains langages intègrent nativement ce principe avec des *quotation* et *antiquotation* par exemple, en Shell ou Perl : `print("Hello $name, 2 + 2 = @{$[2+2]}");`

Note : en examen, pour éviter de perdre du temps, on ne demandera pas de donner le générateur de code en Java, mais **uniquement le schéma de compilation**, sous forme de texte ASCII.

Schéma de compilation des entiers



Compiler.java (extrait)

```
@Override public Void visit(IASTInteger iast, Context context)
throws CompilationException
{
    emit(context.destination.compile());
    emit("ILP_Integer2ILP(");
    emit(iast.getValue().toString());
    emit("); \n");
    return null;
}
...
```

Schéma de compilation des autres constantes

Booléens : true

$\xrightarrow{d}$
true

d ILP_TRUE;

Booléens : false

$\xrightarrow{d}$
false

d ILP_FALSE;

Chaînes

$\xrightarrow{d}$
chaîne

d ILP_String2ILP("escape(chaîne)");

Schéma de compilation

$\xrightarrow{\text{variable}^d}$

$d \text{ variable};$

Compiler.java (extrait)

```
@Override public Void visit(IASTClocalVariable iast, Context context) throws ... {
    emit(context.destination.compile());
    if ( iast.isClosed() ) ...
    else {
        emit(iast.getMangledName());
    }
    emit("; \n");
    return null;
}

@Override public Void visit(IASTCglobalVariable iast, Context context) throws ... {
    emit(context.destination.compile());
    emit(globalVariableEnvironment.getCName(iast));
    emit("; \n");
    return null;
}
```

Le nom de la variable ou de la primitive est converti à un format C.

Schéma de compilation

```


$$\overrightarrow{arg1 + arg2}^d$$



---


{
  ILP_Object tmp1;
  ILP_Object tmp2;
   $\overrightarrow{arg1}^{(tmp1=)}$ 
   $\overrightarrow{arg2}^{(tmp2=)}$ 
   $d$  ILP_Plus(tmp1, tmp2);
}
```

- création de **variables temporaires** `tmp1`, `tmp2`, délimitées par un bloc C `{ ... }`
- calcul des arguments par appel récursif au générateur de code avec **redirection dans les variables temporaires**
ainsi, le code généré pour `arg1` et `arg2` peut être des instructions C qu'il serait impossible d'utiliser en argument de `ILP_Plus`
- calcul final de l'addition dans la destination `d` originale avec `ILP_Plus`

Compiler.java (extrait)

```
@Override public Void visit(IASTbinaryOperation iast, Context context)
throws CompilationException {
    IASTvariable tmp1 = context.newTemporaryVariable();
    IASTvariable tmp2 = context.newTemporaryVariable();
    emit("{ \n");
    emit("  ILP_Object " + tmp1.getMangledName() + "; \n");
    emit("  ILP_Object " + tmp2.getMangledName() + "; \n");
    Context c1 = context.redirect(new AssignDestination(tmp1));
    iast.getLeftOperand().accept(this, c1);
    Context c2 = context.redirect(new AssignDestination(tmp2));
    iast.getRightOperand().accept(this, c2);
    String cName = operatorEnvironment.getBinaryOperator(iast.getOperator());
    emit(context.destination.compile());
    emit(cName);
    emit("(");
    emit(tmp1.getMangledName());
    emit(", ");
    emit(tmp2.getMangledName());
    emit(");\n");
    emit("} \n");
    return null;
}
```

- le nom C de l'opérateur est trouvé dans l'environnement global ([ILP_Plus](#))
- idem pour les opérateurs unaires

Schéma de compilation

```

$$\frac{}{\text{primitive}(arg1, \dots, argN) \xrightarrow{d}}$$


---



```
{
 ILP_Object tmp1;
 ...
 ILP_Object tmpN;
 arg1 \xrightarrow{(tmp1=)}
 ...
 argN \xrightarrow{(tmpN=)}
 d primitiveC(tmp1, ..., tmp2);
}
```


```

- création de **variables temporaires**
- **appel récursif** au générateur de code avec **redirection dans des temporaires**
- appel de la **fonction C primitive** correspondante (cf. environnement global)
- pas de vérification d'**arité** pendant la génération de code, le compilateur C s'en charge !

Compilation des invocations (1/2)

Compiler.java (extrait)

```
@Override public Void visit(IASTInvocation iast, Context context) throws ... {  
    if ( iast instanceof IASTCglobalInvocation )  
        return visit((IASTCglobalInvocation) iast, context);  
    else if ( iast instanceof IASTCcomputedInvocation )  
        return visit((IASTCcomputedInvocation) iast, context);  
    else  
        return visitGeneralInvocation(iast, context);  
}
```

- nécessite un test de type dynamique pour réorienter `IASTInvocation` vers les cas spécifiques `IASTCglobalInvocation`
- les cas `IASTCcomputedInvocation` et `visitGeneralInvocation` seront vus dans ILP3 pour ILP1, nous n'avons que les primitives

Compilation des invocations (2/2)

Compiler.java (extrait)

```
public Void visit(IASTCglobalInvocation iast, Context context) throws ... {
    emit("{ \n");
    IASTexpression[] arguments = iast.getArguments();
    IASTvariable[] tmps = new IASTvariable[arguments.length];
    for ( int i=0 ; i<arguments.length ; i++ ) {
        IASTvariable tmp = context.newTemporaryVariable();
        emit("    ILP_Object " + tmp.getMangledName() + " ; \n");
        tmps[i] = tmp;
    }
    for ( int i=0 ; i<arguments.length ; i++ ) {
        Context c = context.redirect(new AssignDestination(tmps[i]));
        arguments[i].accept(this, c);
    }
    emit(context.destination.compile());
    if ( globalVariableEnvironment.isPrimitive(iast.getFunction()) ) {
        IPrimitive fun = globalVariableEnvironment
            .getPrimitiveDescription(iast.getFunction());
        emit(fun.getCName());
        emit("(");
        for ( int i=0 ; i<arguments.length ; i++ ) {
            emit(tmps[i].getMangledName());
            if ( i < arguments.length-1 ) emit(", ");
        }
    } else ...
    emit(");\n}\n");
    return null;
}
```


Schéma de compilation

$\overrightarrow{\text{if } cond \text{ then } exp1 \text{ else } exp2}^d$

```
{  
  ILP_Object tmp;  
   $\overrightarrow{cond}^{(tmp=)}$   
  if (ILP_isEquivalentToTrue(tmp)) {  $\overrightarrow{exp1}^d$  }  
  else {  $\overrightarrow{exp2}^d$  }  
}
```

- le `if then else` est transformé en `if else C`
- la valeur de la condition apparaît dans une **expression C**
⇒ une variable temporaire `tmp` doit être créée
- les branches `exp1` et `expr2` sont en position d'**instruction**
⇒ la redirection dans une variable temporaire est inutile
- `ILP_isEquivalentToTrue` est utilisé pour l'interprétation en booléen
- idem pour le cas `if then` sans `else`

Compiler.java (extrait)

```
@Override public Void visit(IASTAlternative iast, Context context) throws ... {
    IASTVariable tmp1 = context.newTemporaryVariable();
    emit("{ \n");
    emit("    ILP_Object " + tmp1.getMangledName() + "; \n");
    Context c = context.redirect(new AssignDestination(tmp1));
    iast.getCondition().accept(this, c);
    emit("    if ( ILP_isEquivalentToTrue(");
    emit(tmp1.getMangledName());
    emit(" ) ) {\n");
    iast.getConsequence().accept(this, context);
    if ( iast.isTernary() ) {
        emit("\n    } else {\n");
        iast.getAlternant().accept(this, context);
    }
    emit("\n    }\n");
    return null;
}
```

Schéma de compilation

$$\overrightarrow{\text{expr1}; \dots; \text{exprN} - 1; \text{exprN}}^d$$

{

ILP_Object tmp;

$$\overrightarrow{\text{expr1}}^{(tmp=)};$$

...

$$\overrightarrow{\text{exprN} - 1}^{(tmp=)};$$
$$\overrightarrow{\text{exprN}}^{(tmp=)};$$

d tmp

}

- les expressions expr1 à $\text{exprN} - 1$ sont évaluées dans une **variable temporaire** et leur valeur est **ignorée** ! (on aurait pu aussi utiliser $\overrightarrow{\text{expr}}^{(\text{void})}$)
- la dernière expression est évaluée dans la **variable temporaire** qui est **appliquée à la destination originale** (on aurait pu aussi utiliser $\overrightarrow{\text{exprN}}^d$)

Compiler.java (extrait)

```
@Override public Void visit(IASTsequence iast, Context context) throws ... {
    IASTvariable tmp = context.newTemporaryVariable();
    IASTexpression[] expressions = iast.getExpressions();
    Context c = context.redirect(new AssignDestination(tmp));
    emit("{ \n");
    emit("  ILP_Object " + tmp.getMangledName() + "; \n");
    for ( IASTexpression expr : expressions )
        expr.accept(this, c);
    emit(context.destination.compile());
    emit(tmp.getMangledName());
    emit("; \n} \n");
    return null;
}
```

Schéma de compilation

$$\frac{}{\text{let } x = \text{expr1 in } \text{expr2} \xrightarrow{d}}$$

```
{  
  ILP_Object tmp;  
   $\frac{}{\text{expr1} \xrightarrow{(tmp=)}$ ;  
  {  
    ILP_Object x = tmp;  
     $\frac{}{\text{expr2} \xrightarrow{d}}$   
  }  
}
```

- le résultat est celui d'`expr2`
dans un environnement local où `x` vaut la valeur de `expr1`
 $\implies$ utilisation des blocs C et de la portée lexicale de C!
- extension facile au `let x1 = expr1 and x2 = expr2 ...`
- la temporaire `tmp` semble inutile ici, mais elle deviendra utile en ILP3 pour remplacer `tmp` par `ILP_Value2Box(tmp)`

Compiler.java (extrait)

```
@Override public Void visit(IASTblock iast, Context context) throws ... {
    emit("{ \n");
    IASTbinding[] bindings = iast.getBindings();
    IASTvariable[] tmps = new IASTvariable[bindings.length];
    for ( int i=0 ; i<bindings.length ; i++ ) {
        IASTvariable tmp = context.newTemporaryVariable();
        emit("    ILP_Object " + tmp.getMangledName() + " ; \n");
        tmps[i] = tmp;
    }
    for ( int i=0 ; i<bindings.length ; i++ ) {
        Context c = context.redirect(new AssignDestination(tmps[i]));
        bindings[i].getInitialisation().accept(this, c);
    }
    emit("\n    {\n");
    for ( int i=0 ; i<bindings.length ; i++ ) {
        IASTvariable variable = bindings[i].getVariable();
        emit("        ILP_Object ");
        emit(variable.getMangledName());
        emit(" = ");
        emit(tmps[i].getMangledName());
        emit("; \n");
    }
    iast.getBody().accept(this, context);
    emit("\n    }\n");
    return null;
}
```

```
protected String cProgramPrefix = ""
+ "#include <stdio.h> \n"
+ "#include <stdlib.h> \n"
+ "#include \"ilp.h\" \n\n";

protected String cProgramSuffix = "\n"
+ "static ILP_Object ilp_caught_program () {\n"
+ "    struct ILP_catcher* current_catcher = ILP_current_catcher;\n"
+ "    struct ILP_catcher new_catcher;\n\n"
+ "    if ( 0 == setjmp(new_catcher._jmp_buf) ) {\n"
+ "        ILP_establish_catcher(&new_catcher);\n"
+ "        return ilp_program();\n"
+ "    };\n"
+ "    return ILP_current_exception;\n"
+ "}\n\n"
+ "int main (int argc, char *argv[]) {\n"
+ "    ILP_START_GC; \n"
+ "    ILP_print(ilp_caught_program()); \n"
+ "    ILP_newline(); \n"
+ "    return EXIT_SUCCESS; \n"
+ "} \n\n";
```

Code fixé, qui est copié au début ou à la fin du code C généré :

- `#include` de la bibliothèque C et la bibliothèque d'exécution ILP
- point d'entrée `main` qui appelle `ilp_program` et affiche son résultat
`ilp_caught_program` sert à gérer les exceptions, cf ILP3

Chaîne de compilation

La compilation est un processus complexe qui nécessite :

- plusieurs **étapes successives** :
 - lecture et analyse **syntaxique** du fichier source
 - analyse et transformation de l'**AST**
 - **génération** du code C
 - appel au compilateur et éditeur de lien **C**
 - appel du binaire final généré
- une **configuration** préalable :
 - quelles sont les **primitives** ?
 - quel **analyseur syntaxique** utiliser ?
 - quelles sont les passes d'**optimisation** ? (optionnelles !)

Pour nous aider ILP possède deux classes :

- **CompilerRunner** : regroupe la configuration et la compilation
- **CompilerTest** : mise en œuvre du compilateur et intégration avec JUnit 4

et un script shell :

- **C/compileThenRun.sh** : compile le code C généré puis exécute le binaire

Compiler.java (extrait)

```
public Compiler (IOOperatorEnvironment ioe, IGlobalVariableEnvironment igve) {
    this.operatorEnvironment = ioe;
    this.globalVariableEnvironment = igve;
}

public IASTCprogram normalize(IASTprogram program) throws ... {
    INormalizationFactory nf = new NormalizationFactory();
    Normalizer normalizer = new Normalizer(nf);
    return normalizer.transform(program);
}

public String compile(IASTprogram program) throws ... {
    IASTCprogram newprogram = normalize(program);
    newprogram = optimizer.transform(newprogram); // identité
    // GlobalVariableCollector et FreeVariableCollector omises, cf ILP2-3

    Context context = new Context(NoDestination.NO_DESTINATION);
    StringWriter sw = new StringWriter();
    try {
        out = new BufferedWriter(sw);
        visit(newprogram, context);
        out.flush();
    } catch (IOException exc)
        throw new CompilationException(exc);
    return sw.toString();
}
```

Test JUnit 4 (1/2) : configuration du compilateur

CompilerTest.java (début)

```
@RunWith(Parameterized.class)
public class CompilerTest {
    protected static String scriptCommand = "C/compileThenRun.sh +gc";
    protected static String XMLgrammarFile = "XMLGrammars/grammar1.rng";

    public void configureRunner(CompilerRunner run) throws CompilationException {
        // configuration du parseur
        IASTfactory factory = new ASTfactory();
        run.setILPMLParser(new ILPMLParser(factory));
        IXMLParser xmlparser = new XMLParser(factory);
        xmlparser.setGrammar(new File(XMLgrammarFile));
        run.setXMLParser(xmlparser);

        // configuration du compilateur
        IOperatorEnvironment ioe = new OperatorEnvironment();
        OperatorStuff.fillUnaryOperators(ioe);
        OperatorStuff.fillBinaryOperators(ioe);
        IGlobalVariableEnvironment gve = new GlobalVariableEnvironment();
        GlobalVariableStuff.fillGlobalVariables(gve);
        Compiler compiler = new Compiler(ioe, gve);
        compiler.setOptimizer(new IdentityOptimizer());
        run.setCompiler(compiler);

        // configuration du script de compilation et exécution
        run.setRuntimeScript(scriptCommand);
    }
}
```

Test JUnit (2/2) : lancement du compilateur

CompilerTest.java (fin)

```
protected static String[] samplesDirName = { "SamplesILP1" };
protected static String pattern = "ur?[0-78]\\d*-[123456](gfv)?";

@Test public void processFile()
throws CompilationException, ParseException, IOException {
    CompilerRunner run = new CompilerRunner();
    configureRunner(run);
    run.checkPrintingAndResult(file, run.compileAndRun(file));
}

protected File file;
public CompilerTest(final File file) {
    this.file = file;
}

@Parameters(name = "0")
public static Collection<File[]> data() throws Exception {
    return CompilerRunner.getFileList(samplesDirName, pattern);
}
}
```

Exécution du compilateur et du code compilé

CompilerRunner.java (extrait)

```
public String compileAndRun(File file)
throws ParseException, CompilationException, IOException {
    // lancement du parsing
    IASTprogram program = parser.parse(file);

    // lancement de la compilation vers C
    String compiled = compiler.compile(program);
    File cFile = FileTool.changeSuffix(file, "c");
    FileTool.stuffFile(cFile, compiled);

    // pretty-print du C généré et affichage
    try {
        String indentProgram = "indent " + cFile.getAbsolutePath();
        ProgramCaller pcindent = new ProgramCaller(indentProgram);
        pcindent.run();
    } catch (Exception exc) { }
    System.out.println(FileTool.slurpFile(cFile));

    // lancement du script de compilation et d'exécution
    String compileProgram = "bash " + runtimeScript + " " + cFile.getAbsolutePath();
    ProgramCaller pc = new ProgramCaller(compileProgram);
    pc.setVerbose();
    pc.run();
    assertEquals("Comparing return code", 0, pc.getExitValue());
    return pc.getStdout().trim();
}
```

Script de compilation et d'exécution du code C

C/compileThenRun.sh (simplifié)

```
#!/bin/bash

SRC="$1"
CFLAGS="-Wall -Wno-unused-variable -Wno-unused-but-set-variable -Wno-unused-label -std=c99 -pedantic -g"
DIR='dirname $0'
C_RUNTIME="$DIR/*.c"
LIBS="-lm"
DEFS=""
TARGET='mktemp "/tmp/dlp-XXXXXXXXXX"'

if test "x$GC" != "x"; then
    DEFS="-DWITH_GC $DEFS -I$DIR/gc-8.2.2/include/"
    LIBS="-L$DIR/gc-8.2.2/.libs -lgc"
fi

gcc $CFLAGS -o "$TARGET" -I. -I$DIR $DEFS $C_RUNTIME "$SRC" $OPTS $LIBS && "$TARGET"
```

Tous les fichiers `.c` du répertoire du script sont ajoutés (e.g., `ilp.c`)

Options utiles :

- `+v` : affiche la ligne de compilation avant de l'exécuter
- `+gc` : active BoehmGC

- aspects **statiques** : déterminés par analyse lors de la **compilation**
- aspects **dynamiques** : déterminés par l'**exécution** du code généré
- ILP est un **langage dynamique** : peu d'aspects statiques !
- normalisation de l'AST
 - renommage des variables, classification des variables et des appels
- **représentation des valeurs** ILP en C
- **bibliothèque d'exécution**
- **schéma de compilation**
 - génération de code C
- génération de **temporaires** et **redirection** d'expressions